



FSF3827 Topics in Optimization, 2025
Assignment 2
Due Friday November 14 2025 23.59

Instructions for the assignments

- **Collaborations.** It is fine for students to collaborate on solving the assignment. However, every student needs to submit a report and if you have collaborated with someone you need to mention this clearly in the report. If you collaborate, it is essential that you all solve each task and fully understands all parts.
 - **Using AI / building upon code you found online.** It is fine to use an AI tool or build upon code that you find online. However, you need to fully understand the code and be able to discuss all parts of the code without help from AI. The teacher will ask questions and make sure you understand what the code does. If you use AI, I recommend you ask it to help with writing parts of the code or subroutines and not full programs.
 - The report should have a leading title page where the name, personal number, and e-mail address is clearly stated.
 - You are encouraged to write the reports in Latex.
 - Content
 1. The solutions must be described in enough details that other students (and the teacher) can easily read the report. For example, clearly define variables and sets used in the formulations.
 2. Describe the main parts of the problem formulations. For example, what does the constraints do.
 3. Describe the main parts of the code (what it does and how it works).
 - Everyone needs to upload on the Canvas page of the course no later than by the deadline of the assignment:
 - The report as a pdf file.
 - all the relevant code in a zip file.
 - You will be graded on a pass/fail scale. If the reports is insufficient, you will get second chance to revise the report.
-

In this assignment you will be asked to read a few classical and important papers in mixed-integer programming (MIP), and test some basic concepts.

There are a few questions related to each paper. These are not intended to be difficult, but more a check that you have read the paper. Remember, here the goal is not to just answer the questions but to learn. So, read the papers!

Tasks

1. Read the paper by Land and Doig. This paper first presented the concept of branch-and-bound. This paper is of historical importance, but perhaps not the best paper for actually understanding the different aspects of branch-and-bound.
 - Did this paper introduce the term “branch-and-bound”? If not who did?
 - How is a branching variable chosen in their algorithm?
2. Read the paper “Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning”. This paper gives quite a good overview of the branch-and-bound concept and its main components.
 - Which are the main node selection strategies (or search strategies) that are mentioned in the paper? Briefly discuss why is the order in which you explore nodes important (one paragraph, a longer discussion is not needed).
 - Which branching strategies are mentioned? Is “most fractional” a good branching strategy?
3. Read the paper “Experiments in mixed-integer linear programming”. This is one of the seminal papers in MILP. The paper introduces the concept “pseudo-cost” branching that is one of the most successful branching strategies.
 - Explain, how pseudo-costs are computed and how are they used in branch-and-bound.
4. Read the paper “Branching rules revisited”. This is one of the more influential modern papers on branch-and-bound. The first author Tobias Achterberg, is nowadays leading the algorithm development at Gurobi.
 - What is strong branching (or full strong branching). Explain how it works. Strong branching was presented much earlier, by D. Applegate, R.E. Bixby (“Bi” from Gurobi), V. Chvátal, W. Cook in 1991.
 - Explain how does reliability branching work?
5. Finally, you can test some of the concepts you read about.

Knapsack Problem (KP) is a classical combinatorial optimization problem where a set of items, each with a profit and a weight, must be packed into a knapsack of limited capacity to maximize total profit.

In the *Multi-Knapsack Problem (MKP)*, several knapsacks are available, each with its own capacity. An item can be assigned to at most one knapsack, and the goal is

to maximize the total profit across all knapsacks. The MKP generalizes the standard KP and frequently appears in various applications.

Consider the MKP problem

$$\begin{aligned} &\text{maximize} && \sum_{j=1}^2 \sum_{i=1}^{12} p_i x_{ij} \\ &\text{subject to} && \sum_{i=1}^{13} w_i x_{ij} \leq C_j, \quad j = 1, 2, \\ & && \sum_{j=1}^2 x_{ij} \leq 1, \quad i = 1, \dots, 12, \\ & && x_{ij} \in \{0, 1\}, \quad i = 1, \dots, 13, \quad j = 1, 2, \end{aligned}$$

where

- $m = 2$ is the number of knapsacks, indexed by $j = 1, 2$;
- $n = 12$ is the number of items, indexed by $i = 1, \dots, 12$;
- p_i is the profit of item i , and w_i is the weight of item i ;
- C_j is the capacity of knapsack j ;
- $x_{ij} \in \{0, 1\}$ is a binary variable equal to 1 if item i is assigned to knapsack j .

Item i	Profit p_i	Weight w_i	Item i	Profit p_i	Weight w_i
1	22	5	7	50	10
2	18	6	8	15	4
3	35	8	9	21	6
4	14	4	10	19	5
5	60	11	11	24	7
6	12	3	12	16	5

and the capacities of the knapsacks are $C_1 = 22$, $C_2 = 19$.

- For this problem, you can easily compute a heuristic solution. Just pick some items that seem profitable to fill the knapsacks with. You can use this solution as a bound for in branch-and-bound. (Quickly knowing a feasible solution can help significantly, as parts of the search space with a poor objective value can be discarded).
- Implement a simple branch-and-bound algorithm for solving this problem. You can use Gurobi or any other solver to solve the LP relaxation. Use the best-bound node selection strategy, and start by selecting branching variable as most fractional. How many nodes do you have to explore?
- Change the branching strategy to strong branching. How many nodes do you have to explore?
- From solving the subproblems in strong branching (used for selecting a branching variable), you might also gain other information about the problem. What could that be and how can that knowledge be used (you don't need to implement this, just discuss)?

I was able to solve the problems by exploring less than 100 nodes (but a bit more can also be correct).

Good luck!